Group 4 - CloudNine Systems

Milestone 1: Draft Analysis Report

Status: proposed design for team review. Prepared October 10, 2026. The team statement contains role slots because the confirmed roster has not been supplied.

1. Speculative scenario and organizational problem

CloudNine Systems is a fictional cloud infrastructure provider serving individuals, teams, and organizations. It operates resources in multiple facilities and geographic regions. Clients reserve capacity in advance, run deployments, change their workloads, and pay for compute and ancillary services. Staff coordinate availability, substitutions, support, and billing.

The proposed information system connects requests, actual assignments, measured consumption, and financial responsibility. It should answer who is using a resource at a given time, whether future commitments can be met, how changes affected consumption, and why each account was charged. The project uses synthetic records, not real customer or credential data.

2. Scope and organizational schema

The canonical design has 50 tables across seven modules. The ERD PDF, Mermaid source, JSON schema, and data dictionary describe the same design. Earlier ERD files in docs/ are retained as historical proposals and are not the Milestone 1 submission.

Clients and people: CLIENT represents an individual, team, or organization; USER_ACCOUNT represents a person. CLIENT_MEMBERSHIP stores a person's role over time, while CLIENT_RELATIONSHIP represents organizational and departmental relationships. STAFF extends a user with an operations role. PROJECT groups a client's work. Resource users and the parties responsible for payment are separate concepts.

Infrastructure and history: REGION, AVAILABILITY_ZONE, and DATA_CENTER form a geographic hierarchy. RESOURCE_TYPE describes intended use; RESOURCE identifies independently managed physical or logical assets. RESOURCE_CONFIGURATION and CONFIGURATION_CAPACITY retain changing tiers and numeric capacities. METRIC defines canonical measurement units. RESOURCE_STATE records operational availability, and RESOURCE_RELATIONSHIP records components and dependencies with validity intervals.

Reservations and agreements: RESERVATION identifies the requesting client. RESERVATION_REVISION preserves complete successive request versions. Each version contains RESERVATION_ITEM records, numeric REQUIREMENT_CAPACITY values, and requested RESERVATION_SERVICE records. CONTRACT records agreement terms; RESERVATION_CONTRACT links the applicable agreement without duplicating the agreement's client in the reservation row. COMMITMENT represents deposits and advance commitments.

Deployment and consumption: DEPLOYMENT represents actual activity associated with a reservation and optionally a project. WORKLOAD describes its workloads. RESOURCE_ALLOCATION assigns a specific configuration over an interval; its resource is obtained through that configuration. ALLOCATION_CAPACITY records reserved numeric capacity. DEPLOYMENT_EVENT captures start, stop, pause, resume, scaling, migration, priority and mode changes. SERVICE, RESOURCE_SERVICE and DEPLOYMENT_SERVICE describe ancillary offerings and provisioning. USAGE_RECORD distinguishes estimated from actual metering and retains corrections.

Pricing and charge allocation: RATE_PLAN and effective-dated RATE_RULE records represent resource or service prices, metric units, geographic and tier restrictions, time windows, duration and quantity brackets, and fixed fractional discounts. CONTRACT_RATE_PLAN records which plan applies to an agreement over time. BILLING_ACCOUNT identifies the payer and contact. BILLING_RULE and BILLING_RULE_SHARE define effective sharing agreements. CHARGE records a deployment's monetary obligation or adjustment. CHARGE_USAGE links a metered charge to its measurement and rate. CHARGE_SHARE preserves the actual allocation to payers.

Invoices and payments: INVOICE groups finalized allocated charges for one billing account. INVOICE_LINE associates each charge share with at most one invoice. PAYMENT records receipts or refunds, and PAYMENT_APPLICATION applies a receipt to invoices or deposits. Totals and outstanding balances are derived rather than copied into parent tables.

Access and audit: CREDENTIAL stores only a secure reference or password hash. ACCESS_GRANT defines time-limited permissions on one scope. AUDIT_EVENT captures operational actions and their affected objects. AUTHENTICATION_EVENT separately records successful or failed credential attempts; the credential's owner is derived rather than redundantly stored as an actor.

3. Principal design decisions

Reservation is distinct from allocation: a client can reserve a type and capacity without choosing a physical resource. An unassigned request remains a reservation item. Once a specific configuration is chosen, a planned allocation can be created. Several deployments can draw on one reservation, and each deployment can use resources across zones.

History is retained: a reservation change creates a new revision; reconfiguration creates a new configuration interval; migration closes an allocation and creates a replacement. Changes carry reasons and actors where applicable. This supports later explanations of substitutions and billing disputes.

A logical cluster is a RESOURCE with component relationships. Its administrative data center does not imply that every component occupies that center. Allocation checks expand active component relationships so that allocating a cluster and one of its dedicated components cannot consume the same capacity twice. Shared service capacity may be divided explicitly across allocations.

Operational state is separate from occupancy. A resource may be operational while reserved for tomorrow or in use today. Occupancy is derived from allocation intervals; maintenance and failure are represented in RESOURCE_STATE. Forecasts combine unassigned reservation requirements, planned allocations and operational intervals without counting an assigned requirement twice.

Financial records are historical facts. Rate and sharing changes create new effective versions. Finalized charges and their account shares are immutable; corrections use separately authorized adjustments. A final monetary amount is retained as a posted accounting fact, not treated as a continuously recalculated copy of a formula.

4. Assumptions and integrity constraints

A1. Each facility belongs to one zone and each zone to one region. A resource has one administrative home. Individual users, teams and organizations are client categories; roles and qualifications are assessed manually and documented rather than generated by a scoring engine.

A2. All money is USD and all timestamps are UTC. Effective intervals are half-open [start, end); null end means open-ended. End must exceed start. No real credentials, secrets or personal records are included in example data.

A3. Every published reservation revision has at least one item. Revision numbers are unique within a reservation. Approval determines which revision is operational; the highest approved revision is current, with cancellation handled by reservation status. Items from earlier revisions remain for historical allocations. Staff must explicitly resolve affected future allocations after a revision is approved.

A4. A reservation item may specify at most one location level: region, zone, or data center. Broader locations are derived from the selected level. location_mode is required or preferred; substitutions outside mandatory constraints require a newly approved revision. This avoids redundant stored region and zone identifiers.

A5. Every allocation references exactly one configuration, including planned assignments. The configuration determines the resource, so RESOURCE_ALLOCATION does not repeat resource_id. Its configuration interval must cover the allocation interval. Allocations must satisfy their reservation's approved requirements; their item's reservation must equal the deployment's reservation.

A6. Current configuration and operational-state intervals must not overlap for a resource. Overlapping allocations cannot exceed each capacity metric, and exclusive resources cannot overlap at all. Maintenance and failure intervals block affected capacity. Dependencies and component relationships prohibit self-links and cycles where the relationship semantics require an acyclic graph.

A7. Each usage row references exactly one of allocation_id and deployment_service_id. The metric defines its unit. Quantities are nonnegative; intervals must fall within the referenced allocation or service interval. Corrections create new records linked by supersedes_usage_id and cannot form cycles. Only the latest actual record in a correction chain contributes to finalized usage billing; estimates are displayed separately.

A8. Each rate rule targets exactly one resource type or service. Optional region, tier, hour window, duration bracket and quantity bracket constrain its applicability. Hours use UTC and range from 0 through 23; overnight windows are split into two rules. Numeric bounds are inclusive at the lower end and exclusive at the upper end. Discount fractions lie between 0 and 1. More specific rules receive a higher explicit precedence; equal-precedence competing matches are rejected. Time or threshold crossings are split into separate priced usage slices. Complex bundle eligibility is reviewed manually; its approved discount is represented by an applicable contract plan or an authorized credit.

A9. Each billing rule has exactly one populated scope: client, project or deployment. Effective rules at the same scope cannot overlap. Precedence is deployment, then project, then client. Shares must be positive and total 1. A missing rule requires an explicitly assigned responsible account before a charge is finalized. Cross-organization payment is allowed when authorized, while the project's owning client must match the reservation client.

A10. Ordinary usage charges have exactly one CHARGE_USAGE link; the associated usage must belong to the charge's deployment, and its rate must match the metric and context. Final usage charges require actual records. Provisional charges do not enter invoices. Surcharges and credits require a staff authorizer and a reason. Credits have negative amounts, reference the original charge where applicable, and affect the same deployment. Account shares sum exactly to the posted amount; any rounding residual is assigned deterministically to the largest share.

A11. Invoice lines may reference only final charge shares for the invoice's account. A share appears on at most one invoice. PAYMENT stores a positive amount and payment_type distinguishes receipt from refund. PAYMENT_APPLICATION references exactly one invoice or commitment, matches the payment account, and has a positive amount. Posted applications cannot exceed the payment amount or the target balance. A refund reverses settlement through an audited transaction. Using a funded deposit on an invoice transfers its application atomically instead of counting it twice.

A12. ACCESS_GRANT populates exactly one scope FK. Access is deny-by-default; explicit user permissions are checked at query boundaries. Credentials contain secure references or password hashes, never raw passwords,

tokens or private keys. Human operational events require an actor; automated events may omit one. Authentication failures may have no recognized credential.

A13. Alternate keys include USER_ACCOUNT.email; STAFF.user_id; REGION.name; RESOURCE_TYPE.name; METRIC.name; SERVICE.name; RATE_PLAN.name; CREDENTIAL.credential_reference; PAYMENT.external_reference; RESOURCE(data_center_id, local_identifier); AVAILABILITY_ZONE(region_id, name); DATA_CENTER(zone_id, name); RESERVATION_REVISION(reservation_id, revision_number); CHARGE_SHARE(charge_id, billing_account_id); and CONFIGURATION_CAPACITY(configuration_id, metric_id). Composite primary keys shown in the ERD also enforce uniqueness. Membership and relationship intervals for the same pair and role/type cannot overlap.

A14. Each metric name includes its measurement basis, such as vCPU count, memory capacity in GiB, compute consumption in vCPU-hours or storage consumption in GiB-hours. Unit is a property of the metric. Capacity and consumption metrics are distinct; no implicit unit conversions occur. Metering-to-capacity conversion rules will be documented with the synthetic dataset.

Ordinary PK, FK, UNIQUE and row CHECK constraints enforce local rules. Interval overlap, capacity sums, share totals, mandatory child counts, ownership consistency and lifecycle changes require transactional application checks or database triggers. These are implementation requirements, not claims that a diagram alone enforces them.

5. Third Normal Form rationale

The declared dependencies are the primary or alternate keys determining the remaining attributes of each relation. No other non-key determinants are assumed unless explicitly described below. Scalar notes record one explanatory fact; they are not substitutes for lists of resources, capacities, members or prices.

1NF: every column contains one value. Repeating capacities are separated into CONFIGURATION_CAPACITY, REQUIREMENT_CAPACITY and ALLOCATION_CAPACITY. Members, dependencies, services and billing recipients occupy separate rows rather than arrays or comma-separated lists.

2NF: tables with composite keys store attributes of the entire association. For example, (configuration_id, metric_id) determines capacity; neither identifier alone does. (billing_rule_id, billing_account_id) determines share_fraction. RESOURCE_SERVICE's validity interval belongs to the resource-service-start association. There are no attributes depending only on part of a composite primary key.

3NF: geographical names and parents are stored once in REGION, AVAILABILITY_ZONE and DATA_CENTER. RESOURCE stores only its facility FK. METRIC stores unit; measurements and capacities do not repeat it. An allocation stores configuration_id without resource_id, avoiding configuration_id determining resource_id within the allocation relation. Reservation location columns are mutually exclusive, avoiding a stored facility-to-zone-to-region chain.

RESERVATION_CONTRACT separates agreement selection from the reservation's client. INVOICE_LINE separates invoice membership from CHARGE_SHARE, avoiding invoice_id determining a repeated billing_account_id in that relation. AUTHENTICATION_EVENT derives the owner from CREDENTIAL rather than storing both credential and owner. BILLING_RULE has one scope, so it does not copy client identifiers derivable from project or deployment scope.

CHARGE_USAGE is separated from CHARGE so the charge row does not copy the deployment derivable from a usage reference. A rate rule does not uniquely determine a charge amount: actual posted amounts can reflect authorized adjustments and rounding. Charge amounts and allocated amounts are immutable financial facts. Recalculated totals, availability flags, current priority and account balances are views, not independently

maintained columns.

Under these declared dependencies and XOR rules, the proposed relations satisfy 3NF. Adding new business dependencies during implementation requires repeating the normalization review; introducing surrogate IDs alone would not establish 3NF.

6. Requirements coverage and limitations

Resource types, capacity, tiers and geography map to infrastructure tables. Reconfiguration, dependencies, clustered units, shared services and operational availability map to effective histories and relationships. Advance requests, modifications, delayed assignments and substitutions map to reservation revisions and allocation history. Workloads, scaling, migration, priority and pause/resume map to deployment events, allocations and usage records.

Responsible parties, cost centers, split bills, changing rates, deposits, provisional estimates, surcharges, credits, payments and refunds map to the billing modules and associated integrity rules. Staff contacts, memberships, confidential deployments and operational accountability map to people, access grants and event logs.

The system will support a resource dependency view from RESOURCE_RELATIONSHIP; an interactive zoomable topology is deferred. Allocation optimization, automatic scaling, contractual interpretation, fairness scoring and live metering collection are outside the course implementation. The database retains their decisions and results. SLA and qualification notes are explanatory text; automated evaluation of those terms is not claimed. Service options requiring distinct rates will be modeled as distinct SERVICE offerings; arbitrary nested option documents are excluded.

The web interface can display one or two read-only pages. Authentication infrastructure is not required for this milestone; access metadata remains in the schema because the organizational scenario includes confidentiality. Later work will use at least one quarter of synthetic activity and target eight meaningful aggregate or multi-table queries, covering capacity forecasts, utilization, migration frequency, estimate-versus-actual differences, spending, credits, cost sharing, and outstanding balances. These are plans, not implemented deliverables.

7. Team planning and next steps

The accompanying team statement defines role slots, rotating reviews, workload tracking and support procedures. Actual names, availability and acceptance are pending team confirmation. Before submission, each member should explain one end-to-end example from reservation through deployment and billing, review the assumptions, and confirm repository access. Next, the team will select the course DBMS, translate the logical schema into DDL, validate constraints, generate a realistic dataset and implement queries.

Source basis

Course handouts: Group Project - DB.pdf, seven pages; Group Project_ CloudNine Systems.pdf, five pages. The handouts define the organizational scenario and project expectations. Table names, dependencies, assumptions and scope decisions in this report are this proposed design's interpretation. Milestone 1 follows the three deliverables supplied in the assignment prompt.